

Diacare

Diabetes Prediction & Advisory System



Data Structures Lab

Section: D9

Lab instructor: Hamza Rasheed

Presented By:

Name	Registration No
Muhammad Imran Butt	L1F24BSCS0546
Muhammad Awais	L1F24BSCS0699
Muhammad Zain	L1F24BSCS0670

Faculty of Information Technology

University of Central Punjab

1. Introduction

DiaCare is a console-based C++ application that combines a Machine Learning pipeline with multiple Data Structures to simulate hospital management of diabetic patients. On startup, it loads the Pima Indians Diabetes Dataset, trains a Logistic Regression model, then drops into a menu-driven interface for patient registration, processing, monitoring, and management.

The project splits into two phases. Phase 1 covers the full ML pipeline: data loading, cleaning, training, evaluation. Phase 2 builds the patient management side using Data Structures: Queue, Doubly Linked List, Singly Linked List, Circular Linked List, Stack, and Binary Search Tree. Each structure serves a specific role.

1.1 Goals

- A working patient management system using all required Data Structures.
- ML predictions that actually drive Data Structure behavior, not just sit alongside it.
- A system that resembles a real hospital advisory tool: risk scores, health advice, undo support.

1.2 Technology Stack

Component	Details
Language	C++
Dataset	Pima Indians Diabetes Dataset (768 rows, 8 features)
ML Algorithm	Logistic Regression
Compile Command	<code>g++ main.cpp src/*.cpp -o app.exe -static</code>

2. Project Structure

Path	Description
app.exe / main.cpp	Entry point, starts DiaCareManager and menu loop
Data/diabetes.csv	Pima Indians Diabetes Dataset (768 rows, 8 features)
include/	Header files
BST.h	Binary Search Tree for fast lookup patient
circular_linked_list.h	Circular Linked List, critical patient monitor
config.h	Global constants and configuration
csv_parser.h	CSV reader which loads diabetes.csv into feature arrays
data_printer.h	Console display utilities
diacare_manager.h	Central controller which owns all DS instances and ML pipeline
doubly_linked_list.h	Doubly Linked List , active ward sorted by risk
logistic_regression.h	Gradient descent training and sigmoid prediction
metrics.h	Accuracy, precision, recall, F1 computation
ML_Pipeline.h	Coordinates full ML flow: load → preprocess → train → predict
patient.h	Patient struct + internal SLL of checkup history
preprocessor.h	Imputation, Winsorization, feature engineering, standardization
queue.h	FIFO queue for patient intake
recommendation.h	Maps ML probability to risk level and health advice
stack.h	Undo mechanism, linked-list stack of action snapshots
train_test_split.h	Fisher-Yates shuffle, 80/20 train-test split
src/	Implementation files (mirror of include/)
BST.cpp	Insert, search, 3-case delete, inorder traversal
circular_linked_list.cpp	ML-weighted node insertion, O(1) tail append, cycle logic
csv_parser.cpp	Row-by-row CSV parsing into patient feature vectors
data_printer.cpp	Formatted console output helpers
diacare_manager.cpp	All menu actions, DS coordination, undo routing
doubly_linked_list.cpp	Sorted insert, bidirectional navigation, remove by ID
logistic_regression.cpp	Gradient descent loop, weight update, probability output
metrics.cpp	Confusion matrix derived metrics
ML_Pipeline.cpp	End-to-end pipeline
patient.cpp	Checkup history SLL, risk trajectory slope, destructors
preprocessor.cpp	Fit-on-train, transform-both preprocessing logic
queue.cpp	Enqueue, dequeue, peek, display
recommendation.cpp	Risk level thresholds and clinical advice strings
stack.cpp	Push/pop ActionRecord, deep copy, destructor chain
train_test_split.cpp	Shuffle and split implementation

3. System Architecture

The system has two layers that stay independent but communicate through DiaCareManager. The ML layer handles predictions; the DS layer handles patient management.

```
main.cpp
|
v
DiaCareManager
|-- ML_Pipeline (Phase 1: training + prediction)
|   |-- CSVParser, TrainTestSplit, Preprocessor
|   |-- LogisticRegression (gradient descent)
|   `-- Metrics (accuracy, precision, recall, F1)
|
|-- Queue          (FIFO patient intake)
|-- DoublyLinkedList (active ward, sorted by risk)
|-- CircularLinkedList (critical patient monitor)
|-- Stack          (undo mechanism)
`-- BST            (O(log n) patient search)
```

Key Design Decisions

- Single Source of Truth: one Patient object per patient in heap memory. Every data structure holds a pointer.
- ML drives DS behavior: the probability from the model decides sorted position in the DLL, how many CLL nodes a patient gets, and whether they enter critical monitoring.
- Preprocessor fits on training data only: same stats (medians, caps, mean, std) applied to test data and new inputs. No data leakage.
- Deep copies only in Stack: the undo mechanism stores full snapshots so discharge/re-admission can be reversed cleanly.
- BST for search: replaces $O(n)$ DLL traversal with $O(\log n)$ lookup anywhere a patient needs to be found by ID.

4. Dataset & ML Pipeline

We used the Pima Indians Diabetes Dataset from the kaggle, 768 patient records, 8 features, binary outcome. Split 80/20 (614 train, 154 test) using a Fisher-Yates shuffle.

4.1 Handling Missing Values

Several columns have biologically impossible zero values: zero glucose, zero BMI, zero blood pressure are missing data coded as zero. These were replaced with the non-zero median from training data only.

Column	Action
Glucose, BMI, BloodPressure	Biologically impossible as zero so replaced with non-zero training median
Insulin	48.7% zeros clearly missing so replaced with non-zero training median
SkinThickness	29.6% zeros so replaced with non-zero training median
Pregnancies	Zero is valid (no pregnancies) so left as-is

4.2 Preprocessing Steps

All parameters are computed on training data only, then frozen and applied to test data and new patient inputs at runtime.

1. Imputation: impossible zeros in five columns swapped with training median.
2. Winsorization: all 8 features clipped to 5th–95th percentile from training data to keep gradient descent stable.
3. Feature Engineering: two interaction features added , $\text{Glucose_BMI} = \text{Glucose} \times \text{BMI}$ and $\text{Glucose_Age} = \text{Glucose} / (\text{Age} + 1)$. Original Glucose, Age, and BloodPressure are then dropped. Final model uses 7 features.
4. Standardization: $\text{scaled} = (\text{value} - \text{mean}) / \text{std}$ using training mean and std.

4.3 Logistic Regression

Written entirely from scratch. 1000 epochs of gradient descent, 7 weights + bias. Classification threshold set to 0.46 to push recall higher in a medical context, missing a diabetic patient is a worse error than a false alarm.

Metric	Result
Accuracy	73.2%
Precision	58.6%
Recall	66.7%
Threshold / Epochs / LR	0.46 1000 0.1

5. Data Structures

Queue (Patient Intake)

Standard FIFO queue for patient registration. Only name and age go in on arrival; vitals are entered later when a doctor dequeues and processes them. Matches how real triage works.

Doubly Linked List (Active Ward)

Processed patients go into the DLL sorted descending by diabetes probability. Highest risk always sits at the head. Navigation pointer moves forward (lower risk) and backward (higher risk). When a patient gets a new checkup, they are removed and re-inserted to maintain correct sort order.

Singly Linked List (Checkup History)

Each Patient object owns an internal SLL of CheckupRecord nodes. Every visit appends to the tail. `getRiskTrajectory()` runs to compute the slope of risk change. A slope above 0.10 per visit triggers a RAPID DETERIORATION warning.

Circular Linked List (Critical Monitor)

Patients with probability ≥ 0.60 enter the CLL. The tail points back to the head, creating an infinite loop. ML-weighted insertion gives higher-risk patients more nodes so they appear more often when cycling:

Probability Range	Nodes	Effect
0.60 – 0.75	1	Appears once per cycle
0.76 – 0.89	2	Appears twice per cycle
0.90 and above	3	Appears three times per cycle

Stack (Undo Mechanism)

Every patient action pushes an ActionRecord containing the action type and a deep copy of patient state. Popping the record tells the system exactly how to reverse what happened. The Stack uses a linked list internally so there is no size limit.

Binary Search Tree (Patient Search Index)

Gives $O(\log n)$ patient lookup by ID. Whenever a patient is admitted to the ward, their pointer also goes into the BST keyed by auto-incremented ID. All operations that need to find a patient view history, trajectory, what-if, discharge use `BST.search()` instead of walking the DLL.

6. System Features & Menu

#	Feature	Description
1	About Model	Shows learned weights and test set metrics
2	Register Patient	Creates patient, adds to Queue, records action
3	Process Next Patient	Dequeue, enter vitals, ML predicts, admit to ward
4	View Active Ward	All ward patients sorted by risk probability
5	Navigate Ward	N/P keys move through ward patients
6	New Checkup	Updates vitals, re-runs ML, re-sorts DLL
7	View History	BST finds patient, prints full checkup SLL
8	Risk Trajectory	Slope over last 3 visits, shows trend
9	Monitor Critical	Cycles through CLL, N key advances monitor
10	What-If Simulation	Runs ML on hypothetical vitals, compares to current
11	Discharge Patient	Saves copy for undo, removes from all DS, deletes
12	Undo Last Action	Reverses last action based on ActionRecord type
13	View Queue	Shows all patients waiting to be processed
14	Action History	Lists all recorded actions, most recent first
15	Search by ID	$O(\log n)$ BST search, full patient details
0	Exit	To Exit the program

7. Risk Level System

The recommendation engine maps raw ML probability to a risk level and a set of health advice. Any patient at probability ≥ 0.60 is automatically added to the Critical Monitor (CLL).

8. Conclusion

DiaCare does what it set out to do. The ML side trains a Logistic Regression model entirely from scratch, gets reasonable accuracy on a real dataset, and outputs actual risk probabilities. The Data Structures side builds a functional patient management workflow where each structure has a genuine role.

The part we are most satisfied with is the connection between the two halves. The probability from the ML model directly controls where a patient sits in the DLL, how many nodes they get in the CLL, and whether they enter critical monitoring.